

Reviewer Pack — Coxeter Group Actions on Boolean Functions Bound Communicati...

Entry 3e4938e4b55f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 07:21:03 UTC

Coxeter Group Actions on Boolean Functions Bound Communication Complexity for XOR

Entry ID: 3e4938e4b55f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 07:21:03 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Geometry (Coxeter Groups) **Field B** (complexity object): Complexity Theory: Communication Complexity

Statement:

For every n -ary Boolean function f with communication complexity $C(f)$, the order of the smallest Coxeter group action on f that preserves all its truth table properties is at most $O(C(f)^2)$. Equivalently, there exists a polynomial-time algorithm that, given an n -ary Boolean function f and its communication complexity $C(f)$, computes a Coxeter group action on f with order at most $O(C(f)^2)$.

Rationale (proposer's reasoning):

Coxeter groups are combinatorial structures that have been studied for their geometric and algebraic properties. Their actions on objects can reveal structural information about those objects. By connecting Coxeter group actions to communication complexity, we may uncover new insights into the inherent structure of Boolean functions, potentially leading to improved algorithms for solving problems related to communication complexity.

Taxonomy category: CoxeterGroupActions (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): fe462cc7af22b0fe

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given n -ary Boolean function f , if the smallest Coxeter group action on f that preserves its truth table properties has an order less than or equal to $10 * C(f)^2$, this will be considered supportive evidence.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.90	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter group actions" AND "Boolean functions" AND "communication complexity" - "XOR function" AND "Coxeter groups" AND "algorithmic lower bounds" - "polynomial-time algorithm" AND "Coxeter group action" AND "Boolean function communication complexity"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity(f):
        n = int(math.log2(len(f)))
        max_communication = 0
        for i in range(2**n):
            for j in range(i + 1, 2**n):
                if f[i] != f[j]:
                    max_communication += 1
        return max_communication

    def coxeter_group_action(f):
        n = int(math.log2(len(f)))
        action_order = 0
        while True:
            action_order += 1
            new_f = [f[(i + action_order) % len(f)] for i in range(len(f))]
            if new_f == f:
                return action_order

    n = random.randint(5, 40)
    f = generate_boolean_function(n)
```

```

C_f = communication_complexity(f)
action_order = coxeter_group_action(f)

metric_value = action_order
conjecture_holds = action_order <= 10 * C_f**2
counterexample = "" if conjecture_holds else "Coxeter group action order > 10 * C(f)^2"

return {
    "metric_name": "Coxeter Group Action Order",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    if len(sys.argv) > 1:
        seeds = [int(s) for s in sys.argv[1:]]
    else:
        primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73,
        seeds = random.sample(primes, 30)

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        counterexample = results[first_failing_seed]["counterexample"]
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot confirm whether the smallest Coxeter group action on f that preserves its truth table properties has an order less than or equal to $10 * C(f)^2$. | next: Re-run the test with a longer timeout and ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10554
2	preregistration	ollama_remote	glm4:latest	0	0	5507
3	novelty	ollama_remote	glm4:latest	0	0	4798
4	novelty	ollama_remote	glm4:latest	0	0	5317
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13342
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8072
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8698
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9129
9	judge	ollama_remote	glm4:latest	0	0	14600

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 80018 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/3e4938e4b55f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3e4938e4b55f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3e4938e4b55f.tar.gz` (if generated)